

DOMAIN 5

CONTEXT MANAGEMENT & RELIABILITY

Complete deep-dive for this blueprint domain (15% of exam) of the Claude Certified Architect — Foundations exam: every concept with worked examples, a dedicated traps-and-nuances section explaining why the tempting wrong answers are wrong, a one-page cheat sheet, and a full bank of 116 original practice questions with answers and rationales.

Section	What's inside
1. Cheat Sheet	Golden rule, quick-reference tables, and a scenario decision checklist — read this first and last.
2. Concept Guide	Full narrative coverage of every concept tested in this domain, with worked scenarios.
3. Traps & Nuances	14 recurring wrong-answer patterns: the trap, why it's tempting, and the correct principle.
4. Practice Questions	116 original scenario-based questions, 4 options each, answer marked, rationale included.

This is an independent study aid, not an official Anthropic publication or a reproduction of actual exam questions, which Anthropic does not publish.

1 · CHEAT SHEET

Golden rule: precision-critical facts need a structured, unsummarized home; failure handling needs to be explicit and honest rather than silent. Almost every Domain 5 trap is either lossy summarization of something that shouldn't be lossy, or a silent failure mode dressed up as resilience.

Context reliability essentials

Structured fact block	Transactional facts (amounts, IDs, dates) live outside summarization, sent verbatim every prompt.
Lost-in-the-middle fix	Top-load key findings + explicit section headings, not just shortening.
PostToolUse trimming	Cut oversized tool output down to relevant fields before it enters context.

Reliable vs. unreliable escalation triggers

Reliable	Explicit user request · genuine policy gap · demonstrated inability to progress
Unreliable	Sentiment / tone reads · self-reported confidence · soft classifier scores

Resilience patterns

Idempotency key	Lets a retried call be recognized as the same operation — keep retries, avoid duplication.
Circuit breaker	Stop hammering a clearly-failing dependency; cooldown, then fail fast or fall back.
Exponential backoff	For rate limits — never retry a 429 immediately in a tight loop.
Graceful degradation	Deliver partial value with a clear flag, not all-or-nothing failure.

Decision checklist for any Domain 5 scenario

- A precise value (amount, ID, date) getting vague over a long session? → structured fact block, not “disable summarization.”
- Key finding buried mid-document? → top summary + headings, not blanket shortening.
- Tool returns way more fields than needed? → PostToolUse hook trims it, don't rely on the model to ignore extras.
- Deciding an escalation trigger? → explicit/structural only; reject sentiment, confidence, soft scores.
- A sensitive call might be retried? → idempotency key, not “no retries.”
- A dependency is clearly failing repeatedly? → circuit breaker, not immediate repeated retries.
- Something's unavailable mid-task? → degrade gracefully with a flag, don't fail everything or fake success.

Context Management & Reliability

2 · CONCEPT GUIDE

The smallest domain by weight but a frequent tie-breaker: it covers how context degrades over long sessions, how to keep long documents usable, and — critically — which escalation triggers are trustworthy.

5.1 Progressive summarization loses precision

As a long-running session gets summarized and re-summarized to control context length, precise values erode into vague language: exact figures become “approximately,” specific dates become “recently,” named entities get generalized. This is fine for narrative continuity but dangerous for anything transactional.

Fix: pull transactional facts — amounts, order IDs, exact dates, account numbers — into a **structured block** that lives outside the summarization pipeline and gets included verbatim in every prompt, regardless of how aggressively the surrounding narrative gets compressed.

5.2 The lost-in-the-middle effect

Long-context models reliably attend to the start and end of a long input more than the middle. A 75K-token aggregated research document with the single most important finding buried at token 40,000 is at real risk of being missed or under-weighted. Two structural fixes the exam expects:

- Put the **summary of key findings at the top** of a long document, not just at the end.
- Use **explicit section headings** throughout so the model has clear navigation anchors into the detail, rather than one undifferentiated wall of text.

5.3 Trimming tool output before it reaches the model

Tool outputs accumulate fast across a long agentic session. If a **lookup_order** call returns 40 fields and only 5 are relevant to the current task, sending all 40 into every subsequent prompt wastes context and adds noise the model has to filter itself. A **PostToolUse** hook that trims the raw tool result down to the relevant fields before it enters the conversation keeps the context footprint small without losing signal.

5.4 Escalation triggers — reliable vs. unreliable

This is a high-yield exam topic: knowing which escalation signals are trustworthy enough to build a production decision on.

Reliable triggers	Unreliable triggers (common distractors)
Explicit customer/user request for a human	Sentiment analysis of the conversation
A genuine policy gap the agent has no authority to interpret	The model's self-reported confidence score
Demonstrated inability to make further progress after reasonable attempts	An automatic classifier's soft label with no hard rule behind it

The reasoning behind the split: the reliable column is either an explicit, observable fact (the user asked) or a structurally verifiable condition (there is genuinely no rule to apply, or progress has genuinely stalled after N attempts). The unreliable column depends on the model's own self-assessment or a soft statistical signal — both are exactly the kind of

thing that degrades silently under distribution shift and gives false confidence. Expect the exam to offer all three unreliable options as tempting distractors in the same question.

5.5 Context management checklist

- Keep transactional facts in a structured, unsummarized block included in every prompt.
- Front-load key findings and use section headings in any long aggregated context.
- Trim tool output to relevant fields via PostToolUse hooks rather than passing raw payloads through.
- Escalate on explicit request, genuine policy gaps, or demonstrated stuck-ness — not on sentiment or self-reported confidence.

3 · TRAPS & NUANCES

Each card below names a specific concept, describes the wrong answer the exam is likely to dangle in front of you, explains why that wrong answer feels right, and states the principle that actually resolves it. Read this section right before the exam as a final refresh.

CONCEPT: Progressive summarization

The trap: Disabling summarization entirely to preserve precise values, once the precision-loss problem is noticed.

Why it's tempting: Removing summarization altogether feels like it completely eliminates the risk.

The correct principle: Keep summarization for narrative continuity, but pull transactional facts (amounts, IDs, dates) into a separate structured block that's excluded from summarization and included verbatim in every prompt.

CONCEPT: Lost-in-the-middle effect

The trap: Responding to a buried key finding by uniformly shortening the entire document.

Why it's tempting: "Shorter document = safer" is an intuitive but overly blunt response to the problem.

The correct principle: The fix is structural, not just length-based: front-load key findings in a top summary and add explicit section headings so the model has navigation anchors, regardless of overall length.

CONCEPT: Trimming tool output

The trap: Asking the model to "just ignore" irrelevant fields in a large tool result rather than trimming the result itself.

Why it's tempting: It's the lowest-effort fix — no pipeline change required, just an instruction.

The correct principle: Irrelevant fields should be trimmed programmatically via a PostToolUse hook before they ever enter context — relying on the model to self-filter wastes context and adds noise regardless of instructions.

CONCEPT: Escalation trigger reliability

The trap: Treating a sentiment or tone read on the conversation as a legitimate escalation signal, since it "sounds" like it reflects how the user actually feels.

Why it's tempting: Sentiment analysis produces a number or label that feels like it's measuring something real about the interaction.

The correct principle: Sentiment, self-reported confidence, and soft classifier scores are all internally-generated or statistical signals prone to miscalibration — reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed).

CONCEPT: Structured fact blocks vs. narrative memory

The trap: Assuming a narrative conversation summary is reliable enough for transactional facts as long as it's updated frequently.

Why it's tempting: Frequent updates feel like they should keep the summary accurate.

The correct principle: Narrative summarization is inherently lossy regardless of update frequency — precise facts need a separate, unsummarized structured channel included in every prompt, not a more-often-refreshed narrative.

CONCEPT: Graceful degradation

The trap: Failing an entire task outright when one part of it is unavailable, to avoid delivering an “incomplete” answer.

Why it's tempting: An all-or-nothing failure feels more honest than delivering something partial.

The correct principle: Delivering partial results with a clear note about what's missing preserves value; the failure modes to avoid are silent full failure and silent false confidence, not partial delivery itself.

CONCEPT: Context compaction strategy

The trap: Periodically resetting a long-running session from scratch “to keep things clean.”

Why it's tempting: A fresh start feels like it clears out accumulated mess or confusion.

The correct principle: Deliberate compaction — preserving structured facts and recent context while summarizing or dropping less-relevant older material — retains continuity that a full reset discards.

CONCEPT: Drift monitoring

The trap: Only checking whether a long-running agent has drifted from its original goal when the user complains.

Why it's tempting: Reactive monitoring requires no extra infrastructure and catches the cases that visibly matter to users.

The correct principle: Proactively monitoring against the original goal and constraints catches drift before it silently degrades output — waiting for complaints means damage has already occurred.

CONCEPT: Idempotent retries

The trap: Avoiding retries altogether on a sensitive operation to eliminate the risk of duplicate effects.

Why it's tempting: No retries means no chance of a duplicate — it removes the risk entirely.

The correct principle: The better fix is an idempotency key so a retried call is recognized as the same logical operation — this preserves the resilience benefit of retries without the duplication risk.

CONCEPT: Circuit breakers

The trap: Retrying a clearly failing dependency immediately and repeatedly, since persistence “feels” thorough.

Why it's tempting: Continuing to try feels more resilient and proactive than giving up.

The correct principle: A circuit breaker should trip on a failing dependency — stop calling it for a cooldown period and fail fast or fall back, rather than continuing to hammer something that's clearly down.

CONCEPT: Rate-limit handling

The trap: Retrying immediately in a tight loop the moment a call is rate-limited.

Why it's tempting: Immediate retry feels like the fastest path back to success.

The correct principle: Rate-limited calls need exponential backoff and proactive throttling — immediate, repeated retries usually make the underlying limit worse, not better.

CONCEPT: SLA design

The trap: Skipping explicit timeout and fallback design because the happy path succeeds “most of the time.”

Why it's tempting: If most calls succeed, it can feel like the failure case isn't worth designing for up front.

The correct principle: Meeting a reliability bar requires designing for the failure case explicitly — timeouts, fallback paths, monitoring — not assuming the common case will always hold.

CONCEPT: Error propagation to users

The trap: Showing a raw internal error or stack trace to the end user “for transparency.”

Why it's tempting: Showing the real error can feel more honest than a filtered message.

The correct principle: Good error propagation is honest and actionable without leaking internals — the user should know something went wrong and roughly what to do next, not see raw system detail.

CONCEPT: Caching for reliability

The trap: Periodically regenerating stable, unchanged reference content “to keep it fresh.”

Why it's tempting: Regenerating content feels like good hygiene, similar to refreshing a cache to avoid staleness.

The correct principle: Unnecessary regeneration of genuinely unchanged content risks introducing inconsistency across calls rather than preventing it — caching stable content ensures every call references the identical version.

4 · PRACTICE QUESTIONS (116)

Q1. Over a long session, a refund amount of \$482.17 gets progressively summarized until it appears only as "a refund amount". What's the correct fix?

- A. Disable summarization for the entire session to avoid any loss.
- B. Ask the model to try harder to remember precise values during summarization.
- C. Accept the vague version since narrative continuity matters more than precision.
- D. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.

Correct answer: D. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q2. Over a long session, an order ID of #A-88214 gets progressively summarized until it appears only as "an order". What's the correct fix?

- A. Ask the model to try harder to remember precise values during summarization.
- B. Accept the vague version since narrative continuity matters more than precision.
- C. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- D. Disable summarization for the entire session to avoid any loss.

Correct answer: C. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q3. Over a long session, a meeting scheduled for 2:15 PM on March 14th gets progressively summarized until it appears only as "a meeting sometime soon". What's the correct fix?

- A. Accept the vague version since narrative continuity matters more than precision.
- B. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- C. Disable summarization for the entire session to avoid any loss.
- D. Ask the model to try harder to remember precise values during summarization.

Correct answer: B. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q4. Over a long session, a contract effective date of June 1, 2027 gets progressively summarized until it appears only as "an upcoming effective date". What's the correct fix?

- A. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- B. Disable summarization for the entire session to avoid any loss.
- C. Ask the model to try harder to remember precise values during summarization.
- D. Accept the vague version since narrative continuity matters more than precision.

Correct answer: A. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q5. Over a long session, a customer account number ending in 4471 gets progressively summarized until it appears only as "the customer's account". What's the correct fix?

- A. Disable summarization for the entire session to avoid any loss.
- B. Ask the model to try harder to remember precise values during summarization.
- C. Accept the vague version since narrative continuity matters more than precision.
- D. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.

Correct answer: D. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q6. Over a long session, a shipment tracking number gets progressively summarized until it appears only as "a shipment". What's the correct fix?

- A. Ask the model to try harder to remember precise values during summarization.
- B. Accept the vague version since narrative continuity matters more than precision.
- C. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- D. Disable summarization for the entire session to avoid any loss.

Correct answer: C. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q7. Over a long session, a specific discount of 17% gets progressively summarized until it appears only as "a discount". What's the correct fix?

- A. Accept the vague version since narrative continuity matters more than precision.
- B. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- C. Disable summarization for the entire session to avoid any loss.
- D. Ask the model to try harder to remember precise values during summarization.

Correct answer: B. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q8. Over a long session, an exact patient dosage of 250mg gets progressively summarized until it appears only as "a dosage". What's the correct fix?

- A. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- B. Disable summarization for the entire session to avoid any loss.
- C. Ask the model to try harder to remember precise values during summarization.
- D. Accept the vague version since narrative continuity matters more than precision.

Correct answer: A. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q9. Over a long session, a precise flight number and gate gets progressively summarized until it appears only as "a flight". What's the correct fix?

- A. Disable summarization for the entire session to avoid any loss.
- B. Ask the model to try harder to remember precise values during summarization.
- C. Accept the vague version since narrative continuity matters more than precision.
- D. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.

Correct answer: D. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q10. Over a long session, an exact invoice total of \$3,204.56 gets progressively summarized until it appears only as "an invoice amount". What's the correct fix?

- A. Ask the model to try harder to remember precise values during summarization.
- B. Accept the vague version since narrative continuity matters more than precision.
- C. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- D. Disable summarization for the entire session to avoid any loss.

Correct answer: C. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q11. A 75,000-token aggregated research document has its single most important finding located around token 40,000, with no heading structure. What's the risk, and what's the fix?

- A. Shorten the whole document uniformly with no regard for where the key finding sits.
- B. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- C. Leave the document structure unchanged; length alone isn't a reliability concern.
- D. Move the finding to be repeated only at the very end of the document.

Correct answer: B. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q12. A 60,000-token legal-review compilation has its single most important finding located around the middle third of the document, with no heading structure. What's the risk, and what's the fix?

- A. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- B. Leave the document structure unchanged; length alone isn't a reliability concern.
- C. Move the finding to be repeated only at the very end of the document.
- D. Shorten the whole document uniformly with no regard for where the key finding sits.

Correct answer: A. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q13. A 90,000-token multi-source competitive analysis has its single most important finding located around roughly the halfway point, with no heading structure. What's the risk, and what's the fix?

- A. Leave the document structure unchanged; length alone isn't a reliability concern.
- B. Move the finding to be repeated only at the very end of the document.
- C. Shorten the whole document uniformly with no regard for where the key finding sits.
- D. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.

Correct answer: D. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q14. A 50,000-token customer-feedback aggregation has its single most important finding located around a section buried well past the introduction, with no heading structure. What's the risk, and what's the fix?

- A. Move the finding to be repeated only at the very end of the document.
- B. Shorten the whole document uniformly with no regard for where the key finding sits.
- C. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- D. Leave the document structure unchanged; length alone isn't a reliability concern.

Correct answer: C. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q15. A 100,000-token compiled audit report has its single most important finding located around a finding deep in the middle section, with no heading structure. What's the risk, and what's the fix?

- A. Shorten the whole document uniformly with no regard for where the key finding sits.
- B. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- C. Leave the document structure unchanged; length alone isn't a reliability concern.
- D. Move the finding to be repeated only at the very end of the document.

Correct answer: B. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q16. A 45,000-token merged incident-review document has its single most important finding located around a root-cause note buried mid-document, with no heading structure. What's the risk, and what's the fix?

- A. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- B. Leave the document structure unchanged; length alone isn't a reliability concern.
- C. Move the finding to be repeated only at the very end of the document.
- D. Shorten the whole document uniformly with no regard for where the key finding sits.

Correct answer: A. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q17. An 80,000-token combined market-research report has its single most important finding located around a critical caveat placed mid-document, with no heading structure. What's the risk, and what's the fix?

- A. Leave the document structure unchanged; length alone isn't a reliability concern.
- B. Move the finding to be repeated only at the very end of the document.
- C. Shorten the whole document uniformly with no regard for where the key finding sits.
- D. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.

Correct answer: D. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q18. A 65,000-token merged due-diligence file has its single most important finding located around a key risk factor buried in the middle, with no heading structure. What's the risk, and what's the fix?

- A. Move the finding to be repeated only at the very end of the document.
- B. Shorten the whole document uniformly with no regard for where the key finding sits.
- C. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- D. Leave the document structure unchanged; length alone isn't a reliability concern.

Correct answer: C. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q19. A lookup_order call returns 40 fields when only 5 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Increase the context window instead of trimming the tool output.
- B. ✓ Use a PostToolUse hook to trim the result down to the 5 relevant fields before it enters context.
- C. Pass all fields through unchanged and let the model ignore what it doesn't need.
- D. Call the tool twice and merge the results for redundancy.

Correct answer: B. Trimming at the tool-output layer via a PostToolUse hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q20. A get_customer_profile call returns 35 fields when only 6 are relevant to the current task. What's the best way to reduce the context footprint?

- A. ✓ Use a PostToolUse hook to trim the result down to the 6 relevant fields before it enters context.
- B. Pass all fields through unchanged and let the model ignore what it doesn't need.
- C. Call the tool twice and merge the results for redundancy.
- D. Increase the context window instead of trimming the tool output.

Correct answer: A. Trimming at the tool-output layer via a PostToolUse hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q21. A `fetch_shipment_details` call returns 28 fields when only 4 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Pass all fields through unchanged and let the model ignore what it doesn't need.
- B. Call the tool twice and merge the results for redundancy.
- C. Increase the context window instead of trimming the tool output.
- D. ✓ Use a `PostToolUse` hook to trim the result down to the 4 relevant fields before it enters context.

Correct answer: D. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q22. A `get_employee_record` call returns 32 fields when only 5 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Call the tool twice and merge the results for redundancy.
- B. Increase the context window instead of trimming the tool output.
- C. ✓ Use a `PostToolUse` hook to trim the result down to the 5 relevant fields before it enters context.
- D. Pass all fields through unchanged and let the model ignore what it doesn't need.

Correct answer: C. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q23. A `fetch_transaction` call returns 25 fields when only 4 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Increase the context window instead of trimming the tool output.
- B. ✓ Use a `PostToolUse` hook to trim the result down to the 4 relevant fields before it enters context.
- C. Pass all fields through unchanged and let the model ignore what it doesn't need.
- D. Call the tool twice and merge the results for redundancy.

Correct answer: B. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q24. A `get_property_listing` call returns 30 fields when only 6 are relevant to the current task. What's the best way to reduce the context footprint?

- A. ✓ Use a `PostToolUse` hook to trim the result down to the 6 relevant fields before it enters context.
- B. Pass all fields through unchanged and let the model ignore what it doesn't need.
- C. Call the tool twice and merge the results for redundancy.
- D. Increase the context window instead of trimming the tool output.

Correct answer: A. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q25. A `fetch_patient_record` call returns 45 fields when only 7 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Pass all fields through unchanged and let the model ignore what it doesn't need.
- B. Call the tool twice and merge the results for redundancy.
- C. Increase the context window instead of trimming the tool output.
- D. ✓ Use a `PostToolUse` hook to trim the result down to the 7 relevant fields before it enters context.

Correct answer: D. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q26. A `get_vendor_profile` call returns 27 fields when only 5 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Call the tool twice and merge the results for redundancy.
- B. Increase the context window instead of trimming the tool output.
- C. ✓ Use a `PostToolUse` hook to trim the result down to the 5 relevant fields before it enters context.
- D. Pass all fields through unchanged and let the model ignore what it doesn't need.

Correct answer: C. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q27. Is "the user explicitly asks to speak with a human" a reliable trigger for escalating to a human?

- A. This trigger type is not something a production agent should ever consider.
- B. ✓ Reliable — this is explicit or structurally verifiable.
- C. Unreliable — explicit requests should never be trusted at face value.
- D. Reliability depends only on how the trigger is phrased in code, not its underlying nature.

Correct answer: B. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q28. Is "a sentiment classifier flags the conversation as negative" a reliable trigger for escalating to a human?

- A. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.
- B. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.
- C. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- D. This trigger type is not something a production agent should ever consider.

Correct answer: A. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q29. Is "the agent has made several genuine attempts and cannot proceed further" a reliable trigger for escalating to a human?

- A. Unreliable — explicit requests should never be trusted at face value.
- B. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- C. This trigger type is not something a production agent should ever consider.
- D. ✓ Reliable — this is explicit or structurally verifiable.

Correct answer: D. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q30. Is "the model reports low confidence in its own answer" a reliable trigger for escalating to a human?

- A. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- B. This trigger type is not something a production agent should ever consider.
- C. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.
- D. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.

Correct answer: C. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q31. Is "the request falls into a genuine policy gap the agent has no authority to interpret" a reliable trigger for escalating to a human?

- A. This trigger type is not something a production agent should ever consider.
- B. ✓ Reliable — this is explicit or structurally verifiable.
- C. Unreliable — explicit requests should never be trusted at face value.
- D. Reliability depends only on how the trigger is phrased in code, not its underlying nature.

Correct answer: B. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q32. Is "an automatic soft-label classifier scores the query as "complex"" a reliable trigger for escalating to a human?

- A. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.
- B. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.
- C. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- D. This trigger type is not something a production agent should ever consider.

Correct answer: A. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q33. Is "the user's tone seems frustrated based on word choice alone" a reliable trigger for escalating to a human?

- A. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.
- B. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- C. This trigger type is not something a production agent should ever consider.
- D. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.

Correct answer: D. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q34. Is "the agent has hit a hard, explicit constraint it cannot satisfy through any available action" a reliable trigger for escalating to a human?

- A. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- B. This trigger type is not something a production agent should ever consider.
- C. ✓ Reliable — this is explicit or structurally verifiable.
- D. Unreliable — explicit requests should never be trusted at face value.

Correct answer: C. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q35. Is "a keyword-based heuristic flags certain words as "sensitive"" a reliable trigger for escalating to a human?

- A. This trigger type is not something a production agent should ever consider.
- B. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.
- C. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.
- D. Reliability depends only on how the trigger is phrased in code, not its underlying nature.

Correct answer: B. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q36. Is "the user directly states the automated response isn't resolving their issue and asks for a person" a reliable trigger for escalating to a human?

- A. ✓ Reliable — this is explicit or structurally verifiable.
- B. Unreliable — explicit requests should never be trusted at face value.
- C. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- D. This trigger type is not something a production agent should ever consider.

Correct answer: A. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q37. A long-running support session needs to reliably track a customer's loyalty-tier status and points balance across many turns. What's the most reliable way to maintain this?

- A. Let it live only inside the narrative conversation history, to be re-derived from context each time.
- B. Ask the model to recall it from memory whenever it's needed.
- C. Store it once at the start and never re-check it as the session continues.
- D. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.

Correct answer: D. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q38. A long-running support session needs to reliably track a patient's known allergies and current medications across many turns. What's the most reliable way to maintain this?

- A. Ask the model to recall it from memory whenever it's needed.
- B. Store it once at the start and never re-check it as the session continues.
- C. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.
- D. Let it live only inside the narrative conversation history, to be re-derived from context each time.

Correct answer: C. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q39. A long-running support session needs to reliably track a case's SLA deadline and priority level across many turns. What's the most reliable way to maintain this?

- A. Store it once at the start and never re-check it as the session continues.
- B. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.
- C. Let it live only inside the narrative conversation history, to be re-derived from context each time.
- D. Ask the model to recall it from memory whenever it's needed.

Correct answer: B. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q40. A long-running support session needs to reliably track an order's shipping address and payment method across many turns. What's the most reliable way to maintain this?

- A. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.
- B. Let it live only inside the narrative conversation history, to be re-derived from context each time.
- C. Ask the model to recall it from memory whenever it's needed.
- D. Store it once at the start and never re-check it as the session continues.

Correct answer: A. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q41. A long-running support session needs to reliably track a project's budget cap and approved scope across many turns. What's the most reliable way to maintain this?

- A. Let it live only inside the narrative conversation history, to be re-derived from context each time.
- B. Ask the model to recall it from memory whenever it's needed.
- C. Store it once at the start and never re-check it as the session continues.
- D. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.

Correct answer: D. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q42. A long-running support session needs to reliably track a contract's renewal date and cancellation terms across many turns. What's the most reliable way to maintain this?

- A. Ask the model to recall it from memory whenever it's needed.
- B. Store it once at the start and never re-check it as the session continues.
- C. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.
- D. Let it live only inside the narrative conversation history, to be re-derived from context each time.

Correct answer: C. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q43. In a production agent system, a primary data source becomes temporarily unavailable. What's the reliability-conscious way to handle this?

- A. Retry indefinitely with no fallback or escalation path.
- B. ✓ Fall back to a cached or secondary source and clearly flag that the answer may be based on slightly stale data.
- C. Fail the entire task outright with no partial value delivered.
- D. Proceed as if nothing went wrong, with no indication anything was degraded.

Correct answer: B. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q44. In a production agent system, a specialized tool call fails but a more general tool could partially answer the question. What's the reliability-conscious way to handle this?

- A. ✓ Use the general tool as a fallback and note the reduced precision rather than failing outright.
- B. Fail the entire task outright with no partial value delivered.
- C. Proceed as if nothing went wrong, with no indication anything was degraded.
- D. Retry indefinitely with no fallback or escalation path.

Correct answer: A. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q45. In a production agent system, a subagent times out but partial results were already gathered. What's the reliability-conscious way to handle this?

- A. Fail the entire task outright with no partial value delivered.
- B. Proceed as if nothing went wrong, with no indication anything was degraded.
- C. Retry indefinitely with no fallback or escalation path.
- D. ✓ Return the partial results with a clear note about what's missing, rather than discarding everything.

Correct answer: D. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q46. In a production agent system, the primary model is temporarily rate-limited. What's the reliability-conscious way to handle this?

- A. Proceed as if nothing went wrong, with no indication anything was degraded.
- B. Retry indefinitely with no fallback or escalation path.
- C. ✓ Queue or gracefully delay the request with a clear status, rather than silently dropping it.
- D. Fail the entire task outright with no partial value delivered.

Correct answer: C. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q47. In a production agent system, an external API used for enrichment is down. What's the reliability-conscious way to handle this?

- A. Retry indefinitely with no fallback or escalation path.
- B. ✓ Proceed with the core task using only the data that's actually available, flagging the missing enrichment.
- C. Fail the entire task outright with no partial value delivered.
- D. Proceed as if nothing went wrong, with no indication anything was degraded.

Correct answer: B. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q48. In a production agent system, a required field can't be retrieved from any available source. What's the reliability-conscious way to handle this?

- A. ✓ Explicitly mark the field as unavailable rather than fabricating a plausible-looking value.
- B. Fail the entire task outright with no partial value delivered.
- C. Proceed as if nothing went wrong, with no indication anything was degraded.
- D. Retry indefinitely with no fallback or escalation path.

Correct answer: A. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q49. In a production agent system, a downstream service needed for final formatting is unavailable. What's the reliability-conscious way to handle this?

- A. Fail the entire task outright with no partial value delivered.
- B. Proceed as if nothing went wrong, with no indication anything was degraded.
- C. Retry indefinitely with no fallback or escalation path.
- D. ✓ Return the unformatted result with a note, rather than blocking the entire task on formatting.

Correct answer: D. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q50. In a production agent system, a validation service is temporarily down. What's the reliability-conscious way to handle this?

- A. Proceed as if nothing went wrong, with no indication anything was degraded.
- B. Retry indefinitely with no fallback or escalation path.
- C. ✓ Hold the result for validation once the service returns, rather than skipping validation silently.
- D. Fail the entire task outright with no partial value delivered.

Correct answer: C. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q51. For a customer-support session that can run for hours across many back-and-forth turns, how should context be managed as the session grows very long?

- A. Keep every single turn verbatim forever regardless of context-window limits.
- B. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- C. Let the context window fill up naturally and let truncation happen wherever it falls.
- D. Reset the session from scratch periodically with no continuity at all.

Correct answer: B. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q52. For a long research task that accumulates many tool results over time, how should context be managed as the session grows very long?

- A. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- B. Let the context window fill up naturally and let truncation happen wherever it falls.
- C. Reset the session from scratch periodically with no continuity at all.
- D. Keep every single turn verbatim forever regardless of context-window limits.

Correct answer: A. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q53. For a multi-day project-tracking assistant with a large accumulated history, how should context be managed as the session grows very long?

- A. Let the context window fill up naturally and let truncation happen wherever it falls.
- B. Reset the session from scratch periodically with no continuity at all.
- C. Keep every single turn verbatim forever regardless of context-window limits.
- D. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.

Correct answer: D. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q54. For an extended code-review session touching many files over time, how should context be managed as the session grows very long?

- A. Reset the session from scratch periodically with no continuity at all.
- B. Keep every single turn verbatim forever regardless of context-window limits.
- C. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- D. Let the context window fill up naturally and let truncation happen wherever it falls.

Correct answer: C. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q55. For a long-running incident-response session with a growing timeline of events, how should context be managed as the session grows very long?

- A. Keep every single turn verbatim forever regardless of context-window limits.
- B. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- C. Let the context window fill up naturally and let truncation happen wherever it falls.
- D. Reset the session from scratch periodically with no continuity at all.

Correct answer: B. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q56. For a multi-session onboarding assistant that should recall earlier sessions, how should context be managed as the session grows very long?

- A. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- B. Let the context window fill up naturally and let truncation happen wherever it falls.
- C. Reset the session from scratch periodically with no continuity at all.
- D. Keep every single turn verbatim forever regardless of context-window limits.

Correct answer: A. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q57. For a customer-support agent that has been running for an unusually long session, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Assume long sessions are always as reliable as short ones with no extra monitoring.
- B. Only check for drift if the user explicitly complains.
- C. Restart the session automatically at a fixed time interval with no regard for actual state.
- D. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.

Correct answer: D. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q58. For a research agent that has gone through many rounds of tool calls, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Only check for drift if the user explicitly complains.
- B. Restart the session automatically at a fixed time interval with no regard for actual state.
- C. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.
- D. Assume long sessions are always as reliable as short ones with no extra monitoring.

Correct answer: C. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q59. For a long-running data-cleanup agent operating over many hours, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Restart the session automatically at a fixed time interval with no regard for actual state.
- B. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.
- C. Assume long sessions are always as reliable as short ones with no extra monitoring.
- D. Only check for drift if the user explicitly complains.

Correct answer: B. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q60. For an extended code-migration agent working across a large codebase, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.
- B. Assume long sessions are always as reliable as short ones with no extra monitoring.
- C. Only check for drift if the user explicitly complains.
- D. Restart the session automatically at a fixed time interval with no regard for actual state.

Correct answer: A. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q61. For a multi-day monitoring agent that continuously ingests new data, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Assume long sessions are always as reliable as short ones with no extra monitoring.
- B. Only check for drift if the user explicitly complains.
- C. Restart the session automatically at a fixed time interval with no regard for actual state.
- D. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.

Correct answer: D. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q62. For a long conversational assistant session spanning many topic changes, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Only check for drift if the user explicitly complains.
- B. Restart the session automatically at a fixed time interval with no regard for actual state.
- C. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.
- D. Assume long sessions are always as reliable as short ones with no extra monitoring.

Correct answer: C. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q63. A financial-approval agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. Log only failures, since successful runs don't need a trace.
- B. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- C. Log only the final output, with no record of how the agent got there.
- D. Keep no logs at all to minimize storage and complexity.

Correct answer: B. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q64. A medical-information assistant operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- B. Log only the final output, with no record of how the agent got there.
- C. Keep no logs at all to minimize storage and complexity.
- D. Log only failures, since successful runs don't need a trace.

Correct answer: A. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q65. A legal-document-review agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. Log only the final output, with no record of how the agent got there.
- B. Keep no logs at all to minimize storage and complexity.
- C. Log only failures, since successful runs don't need a trace.
- D. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.

Correct answer: D. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q66. A customer-data-access agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. Keep no logs at all to minimize storage and complexity.
- B. Log only failures, since successful runs don't need a trace.
- C. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- D. Log only the final output, with no record of how the agent got there.

Correct answer: C. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q67. An hr-decision-support agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. Log only failures, since successful runs don't need a trace.
- B. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- C. Log only the final output, with no record of how the agent got there.
- D. Keep no logs at all to minimize storage and complexity.

Correct answer: B. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q68. A compliance-monitoring agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- B. Log only the final output, with no record of how the agent got there.
- C. Keep no logs at all to minimize storage and complexity.
- D. Log only failures, since successful runs don't need a trace.

Correct answer: A. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q69. A reliability-conscious agent system includes a payment-confirmation step that could be retried after a network blip that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Avoid retries altogether, even for clearly transient failures.
- B. Let the duplicate effect happen and reconcile it with a manual process afterward.
- C. Retry with a brand-new identifier each time to be safe.
- D. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.

Correct answer: D. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q70. A reliability-conscious agent system includes an order-submission step that might be retried after an ambiguous timeout that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Let the duplicate effect happen and reconcile it with a manual process afterward.
- B. Retry with a brand-new identifier each time to be safe.
- C. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.
- D. Avoid retries altogether, even for clearly transient failures.

Correct answer: C. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q71. A reliability-conscious agent system includes a notification-send step that could fire twice after a retry that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Retry with a brand-new identifier each time to be safe.
- B. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.
- C. Avoid retries altogether, even for clearly transient failures.
- D. Let the duplicate effect happen and reconcile it with a manual process afterward.

Correct answer: B. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q72. A reliability-conscious agent system includes a resource-provisioning step that might be invoked twice due to a retry that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.
- B. Avoid retries altogether, even for clearly transient failures.
- C. Let the duplicate effect happen and reconcile it with a manual process afterward.
- D. Retry with a brand-new identifier each time to be safe.

Correct answer: A. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q73. A reliability-conscious agent system includes an inventory-adjustment step that could be retried after a partial failure that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Avoid retries altogether, even for clearly transient failures.
- B. Let the duplicate effect happen and reconcile it with a manual process afterward.
- C. Retry with a brand-new identifier each time to be safe.
- D. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.

Correct answer: D. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q74. A reliability-conscious agent system includes a record-creation step that might be retried after an unclear response that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Let the duplicate effect happen and reconcile it with a manual process afterward.
- B. Retry with a brand-new identifier each time to be safe.
- C. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.
- D. Avoid retries altogether, even for clearly transient failures.

Correct answer: C. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q75. A downstream enrichment api that has started failing repeatedly. What reliability pattern helps prevent this from cascading into a larger outage?

- A. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.
- B. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- C. Keep retrying every call immediately with no pause, regardless of the failure rate.
- D. Wait indefinitely for the dependency to recover with no fallback in the meantime.

Correct answer: B. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q76. A subagent type that has failed on the last several consecutive attempts. What reliability pattern helps prevent this from cascading into a larger outage?

- A. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- B. Keep retrying every call immediately with no pause, regardless of the failure rate.
- C. Wait indefinitely for the dependency to recover with no fallback in the meantime.
- D. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.

Correct answer: A. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q77. An external search provider returning errors on most recent calls. What reliability pattern helps prevent this from cascading into a larger outage?

- A. Keep retrying every call immediately with no pause, regardless of the failure rate.
- B. Wait indefinitely for the dependency to recover with no fallback in the meantime.
- C. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.
- D. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.

Correct answer: D. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q78. A database that has been timing out on the majority of recent queries. What reliability pattern helps prevent this from cascading into a larger outage?

- A. Wait indefinitely for the dependency to recover with no fallback in the meantime.
- B. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.
- C. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- D. Keep retrying every call immediately with no pause, regardless of the failure rate.

Correct answer: C. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q79. A third-party notification service failing on nearly every recent call. What reliability pattern helps prevent this from cascading into a larger outage?

- A. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.
- B. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- C. Keep retrying every call immediately with no pause, regardless of the failure rate.
- D. Wait indefinitely for the dependency to recover with no fallback in the meantime.

Correct answer: B. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q80. An internal microservice returning errors far more often than usual. What reliability pattern helps prevent this from cascading into a larger outage?

- A. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- B. Keep retrying every call immediately with no pause, regardless of the failure rate.
- C. Wait indefinitely for the dependency to recover with no fallback in the meantime.
- D. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.

Correct answer: A. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q81. A research agent that issues many rapid tool calls in a short window starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Retry every rate-limited call immediately in a tight loop until it succeeds.
- B. Ignore rate-limit responses and keep sending calls at the same pace.
- C. Abandon the task entirely the first time a rate limit is hit.
- D. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.

Correct answer: D. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q82. A bulk-processing agent iterating over thousands of records starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Ignore rate-limit responses and keep sending calls at the same pace.
- B. Abandon the task entirely the first time a rate limit is hit.
- C. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.
- D. Retry every rate-limited call immediately in a tight loop until it succeeds.

Correct answer: C. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q83. A multi-subagent system where several subagents call the same api concurrently starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Abandon the task entirely the first time a rate limit is hit.
- B. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.
- C. Retry every rate-limited call immediately in a tight loop until it succeeds.
- D. Ignore rate-limit responses and keep sending calls at the same pace.

Correct answer: B. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q84. A monitoring agent that polls an external service frequently starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.
- B. Retry every rate-limited call immediately in a tight loop until it succeeds.
- C. Ignore rate-limit responses and keep sending calls at the same pace.
- D. Abandon the task entirely the first time a rate limit is hit.

Correct answer: A. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q85. A data-enrichment pipeline calling a rate-limited third-party api starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Retry every rate-limited call immediately in a tight loop until it succeeds.
- B. Ignore rate-limit responses and keep sending calls at the same pace.
- C. Abandon the task entirely the first time a rate limit is hit.
- D. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.

Correct answer: D. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q86. A notification-fanout agent sending many messages in a short burst starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Ignore rate-limit responses and keep sending calls at the same pace.
- B. Abandon the task entirely the first time a rate limit is hit.
- C. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.
- D. Retry every rate-limited call immediately in a tight loop until it succeeds.

Correct answer: C. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q87. A customer-facing support agent with a committed response-time sla needs to meet a defined reliability bar. What should the design account for?

- A. Treat any SLA miss identically to a total system failure with no graceful handling.
- B. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- C. Assume the agent will always respond in time with no explicit timeout or fallback design.
- D. Set no monitoring and only find out about SLA misses from user complaints.

Correct answer: B. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q88. An internal agent whose output feeds a time-sensitive business process needs to meet a defined reliability bar. What should the design account for?

- A. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- B. Assume the agent will always respond in time with no explicit timeout or fallback design.
- C. Set no monitoring and only find out about SLA misses from user complaints.
- D. Treat any SLA miss identically to a total system failure with no graceful handling.

Correct answer: A. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q89. A production monitoring agent that must reliably detect incidents needs to meet a defined reliability bar. What should the design account for?

- A. Assume the agent will always respond in time with no explicit timeout or fallback design.
- B. Set no monitoring and only find out about SLA misses from user complaints.
- C. Treat any SLA miss identically to a total system failure with no graceful handling.
- D. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.

Correct answer: D. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q90. A compliance-checking agent with a required turnaround time needs to meet a defined reliability bar. What should the design account for?

- A. Set no monitoring and only find out about SLA misses from user complaints.
- B. Treat any SLA miss identically to a total system failure with no graceful handling.
- C. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- D. Assume the agent will always respond in time with no explicit timeout or fallback design.

Correct answer: C. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q91. A real-time fraud-screening agent gating live transactions needs to meet a defined reliability bar. What should the design account for?

- A. Treat any SLA miss identically to a total system failure with no graceful handling.
- B. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- C. Assume the agent will always respond in time with no explicit timeout or fallback design.
- D. Set no monitoring and only find out about SLA misses from user complaints.

Correct answer: B. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q92. An agent embedded in a checkout flow with strict latency expectations needs to meet a defined reliability bar. What should the design account for?

- A. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- B. Assume the agent will always respond in time with no explicit timeout or fallback design.
- C. Set no monitoring and only find out about SLA misses from user complaints.
- D. Treat any SLA miss identically to a total system failure with no graceful handling.

Correct answer: A. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q93. A long-running multi-step migration task is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. Restart the entire task from the very beginning after any interruption.
- B. Keep all state only in memory with no persistence at all.
- C. Abandon the task entirely if it's interrupted partway through.
- D. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.

Correct answer: D. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q94. A multi-hour data-processing job is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. Keep all state only in memory with no persistence at all.
- B. Abandon the task entirely if it's interrupted partway through.
- C. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.
- D. Restart the entire task from the very beginning after any interruption.

Correct answer: C. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q95. An extended multi-turn customer-support session is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. Abandon the task entirely if it's interrupted partway through.
- B. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.
- C. Restart the entire task from the very beginning after any interruption.
- D. Keep all state only in memory with no persistence at all.

Correct answer: B. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q96. A long research task with many completed subtasks is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.
- B. Restart the entire task from the very beginning after any interruption.
- C. Keep all state only in memory with no persistence at all.
- D. Abandon the task entirely if it's interrupted partway through.

Correct answer: A. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q97. A multi-stage document-review pipeline is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. Restart the entire task from the very beginning after any interruption.
- B. Keep all state only in memory with no persistence at all.
- C. Abandon the task entirely if it's interrupted partway through.
- D. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.

Correct answer: D. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q98. A long-running batch-classification job is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. Keep all state only in memory with no persistence at all.
- B. Abandon the task entirely if it's interrupted partway through.
- C. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.
- D. Restart the entire task from the very beginning after any interruption.

Correct answer: C. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q99. A production agent system currently has a raw stack trace being shown directly to an end user. What's the better design for error propagation to the user?

- A. Suppress all error indication so the user believes everything succeeded.
- B. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- C. Show the raw internal error directly to the user unfiltered.
- D. Always show a generic message with no distinction between different failure types.

Correct answer: B. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q100. A production agent system currently has a generic "something went wrong" message with no actionable detail for the user. What's the better design for error propagation to the user?

- A. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- B. Show the raw internal error directly to the user unfiltered.
- C. Always show a generic message with no distinction between different failure types.
- D. Suppress all error indication so the user believes everything succeeded.

Correct answer: A. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q101. A production agent system currently has an internal error code surfaced with no plain-language explanation. What's the better design for error propagation to the user?

- A. Show the raw internal error directly to the user unfiltered.
- B. Always show a generic message with no distinction between different failure types.
- C. Suppress all error indication so the user believes everything succeeded.
- D. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.

Correct answer: D. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q102. A production agent system currently has a silent failure where the user sees a successful-looking response that's actually incomplete. What's the better design for error propagation to the user?

- A. Always show a generic message with no distinction between different failure types.
- B. Suppress all error indication so the user believes everything succeeded.
- C. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- D. Show the raw internal error directly to the user unfiltered.

Correct answer: C. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q103. A production agent system currently has an error message that reveals internal system details irrelevant or confusing to the user. What's the better design for error propagation to the user?

- A. Suppress all error indication so the user believes everything succeeded.
- B. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- C. Show the raw internal error directly to the user unfiltered.
- D. Always show a generic message with no distinction between different failure types.

Correct answer: B. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q104. A production agent system currently has a timeout that's presented to the user identically to a definitive failure. What's the better design for error propagation to the user?

- A. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- B. Show the raw internal error directly to the user unfiltered.
- C. Always show a generic message with no distinction between different failure types.
- D. Suppress all error indication so the user believes everything succeeded.

Correct answer: A. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q105. A production system relies on a policy document referenced identically across thousands of support interactions across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Re-send a freshly retrieved copy of the context on every call with no caching.
- B. Let each call use whatever version of the context happens to be locally cached by chance.
- C. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- D. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.

Correct answer: D. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q106. A production system relies on a large product catalog referenced repeatedly across many sessions across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Let each call use whatever version of the context happens to be locally cached by chance.
- B. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- C. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.
- D. Re-send a freshly retrieved copy of the context on every call with no caching.

Correct answer: C. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q107. A production system relies on a legal-terms reference used consistently across many contract reviews across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- B. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.
- C. Re-send a freshly retrieved copy of the context on every call with no caching.
- D. Let each call use whatever version of the context happens to be locally cached by chance.

Correct answer: B. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q108. A production system relies on a knowledge base referenced the same way across many customer queries across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.
- B. Re-send a freshly retrieved copy of the context on every call with no caching.
- C. Let each call use whatever version of the context happens to be locally cached by chance.
- D. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.

Correct answer: A. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q109. A production system relies on a system-wide style guide referenced across many content-generation calls across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Re-send a freshly retrieved copy of the context on every call with no caching.
- B. Let each call use whatever version of the context happens to be locally cached by chance.
- C. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- D. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.

Correct answer: D. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q110. A production system relies on a compliance checklist referenced identically across many audit tasks across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Let each call use whatever version of the context happens to be locally cached by chance.
- B. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- C. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.
- D. Re-send a freshly retrieved copy of the context on every call with no caching.

Correct answer: C. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q111. Before a multi-agent research pipeline goes into production, how should its reliability under failure conditions be validated?

- A. Test only the successful path and skip testing any failure scenarios.
- B. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- C. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- D. Only discover failure behavior when it happens for real in production.

Correct answer: B. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q112. Before a customer-support orchestration system goes into production, how should its reliability under failure conditions be validated?

- A. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- B. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- C. Only discover failure behavior when it happens for real in production.
- D. Test only the successful path and skip testing any failure scenarios.

Correct answer: A. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q113. Before a document-processing pipeline goes into production, how should its reliability under failure conditions be validated?

- A. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- B. Only discover failure behavior when it happens for real in production.
- C. Test only the successful path and skip testing any failure scenarios.
- D. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.

Correct answer: D. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q114. Before a payment-approval workflow goes into production, how should its reliability under failure conditions be validated?

- A. Only discover failure behavior when it happens for real in production.
- B. Test only the successful path and skip testing any failure scenarios.
- C. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- D. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.

Correct answer: C. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q115. Before a data-migration agent system goes into production, how should its reliability under failure conditions be validated?

- A. Test only the successful path and skip testing any failure scenarios.
- B. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- C. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- D. Only discover failure behavior when it happens for real in production.

Correct answer: B. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q116. Before an incident-response automation system goes into production, how should its reliability under failure conditions be validated?

- A. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- B. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- C. Only discover failure behavior when it happens for real in production.
- D. Test only the successful path and skip testing any failure scenarios.

Correct answer: A. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

End of Domain 5 deep-dive. This is an independent study aid; verify current exam logistics and blueprint details against your Claude Partner Network portal before sitting the exam.